



Race Vision

Description

RaceVision is a deep learning pipeline for lap-by-lap prediction of Formula 1 races.

It uses historical race data from the Ergast Motor Racing Database to build time-series datasets and trains an LSTM model to forecast driver positions, lap times, and pit-stop events for all 20 grid starters.

Files structure

RaceVision/

— scripts/	
— 1-DataPreparation.py	# Converts raw Ergast DB CSVs to per-race/lap CSVs
— 2-RacePredictionPipeline.py	# Builds embeddings, .npy datasets, and defines & trains LSTM
model	
— db/	# Raw and processed data directory (user-specified)
— races.csv	# Metadata: race calendar
— lap_times.csv	# Raw lap-time logs
— results.csv	# Race results per driver
— qualifying.csv	# Qualifying session times
— pit_stops.csv	# Pit-stop events (from 2012)
— driver_standings.csv	# Championship standings per round
— constructor_standings.csv	# Constructor standings per round
— races/	# Generated per-race CSVs by script 1
— <year>/race<raceId>.csv	
— drivers_short.csv	# Unique driverId listing
— races_npy/	# Generated .npy inputs/outputs by script 2
— <year>/{idx}_in.npy	
— {idx}_exp.npy	
— sd/	# Model checkpoints and final weights

Data gathreing

Source

Our data are from Ergast DB : <http://ergast.com/mrd/db/>

Ergast files used :

- races.csv : race metadata (year, round, circuit, etc.)
- driver_standings.csv : standings after each round
- constructor_standings.csv : standings after each round
- qualifying.csv : Q1/Q2/Q3 lap times
- pit_stops.csv : pit stop events (from 2012 onward)
- lap_times.csv : every driver's lap-by-lap time
- results.csv : final finishing order and status per driver per race

Data preparation

Wrap-up structure

Each row in the per-race, per-lap CSV represents one lap (lap 0 is the pre-race “qualifying” snapshot, laps 1...N are real race laps) and contains:

Dataframe Structure		
circuitID	ID of the circuit where the race is held	integer
For each of the 20 grid slots (k = 1...20), seven fields:		
driverId{k}	The driver's unique ID	integer
driverStanding{k}	That driver's championship standing before this race	integer
constructorStanding{k}	That driver's constructor's standing before this race	integer
position{k}	Driver's track position at the end of this lap (1...20; 21 = retired...)	integer
inPit{k}	1 if the driver pitted on this lap; otherwise 0	integer
status{k}	Status code (0 = racing, non-zero = retirement/other)	integer
laptime{k}	Lap time in seconds (or qualifying time on lap 0; 0 if missing/retired)	float or 0

One-hot

Data normalization

Numeric value	0	1	2	3	4	5	6	7	8	9	10
One-hot	0	0	0	0	0	0	1	0	0	0	0

Advantages

1 - Ordinality

Integers imply order :

driverId 10 may be understood as “twice” driverId 5 or “more” in some monotonic sense.

2 - Sparse, High-Dimensional Representations

Sparsity (mostly zeros, one “1”) leads to more stable gradients and less interference between categories.

3 - Consistent Scaling

Mixing integers of wildly different ranges (e.g. driverId up to 130, lap time ~100 s, positions 1–21) may be tricky to normalize.

One-hot entirely sidesteps that, since all features are in {0,1}.

In practice for my script

Circuit ID, driver ID and position are all categorical.

By embedding each as a one-hot of length 130, 130, or 21 respectively, I give my LSTM a clean, non-ordinal signal.

The network can then learn—for example—that driver 7 and driver 12 have entirely different learned interactions with lap percentage or pit flags, without confusing them because of a numeric scale.

Data setup

.npy input

Each row in the numpy file represents one lap (lap 0 is the pre-race “qualifying” snapshot, laps 1...N are real race laps) and contains:

Dataframe Structure

circuitID	ID of the circuit where the race is held	One-hot 130d
Lap number	lap in race percentage progress	Float
For each of the 20 grid slots (k = 1...20), seven fields:		
driverId{k}	The driver's unique ID	One-hot 130d
driverStanding{k}	That driver's championship standing before this race	One-hot 3d
constructorStanding{k}	That driver's constructor's standing before this race	One-hot 2d
position{k}	Driver's track position at the end of this lap (1...20; 21 = retired...)	One-hot 21d
inPit{k}	1 if the driver pitted on this lap; otherwise 0	Binary
status{k}	Status code (0 = racing, non-zero = retirement/other)	One-hot 6d
laptime{k}	Lap time in seconds (or qualifying time on lap 0; 0 if missing/retired)	One-hot 32d

Data setup

.npy output

Each row corresponds to the predicted position of each driver for lap $i+1$ (given inputs from lap i).

Dataframe Structure

For each of the 20 grid slots ($k = 1 \dots 20$), seven fields:

position{k}	Driver's track position at the end of this lap (1...20; 21 = retired...)	One-hot 21d
laps-to-pit{k}	1 if the driver pitted on this lap; otherwise 0	Binary
status{k}	Status code (0 = racing, non-zero = retirement/other)	One-hot 6d
laptime{k}	Lap time in seconds (or qualifying time on lap 0; 0 if missing/retired)	One-hot 32d

Model choice

What we expect from our model :

- **Remember what happened during previous laps**

- Races can be 50+ laps long, and key events (pit-stops, tyre-degradation) often happen many laps earlier.
- The driver leading the championship may not suddenly become slow because of an accident.

- **Length of variables may not be fix**

- Different Grands Prix have different numbers of laps

- **Manage the volume of datas**

- $20 \text{ drivers} \times \sim 50 \text{ laps} \times \sim 20 \text{ races} \times \sim 20 \text{ years}$

- **Different types of variables**

- We use one-hots, scalars and binaries

- **Easy to handle**

- Time limitation is key to respect the deadline

Why LSTM Model

What LSTM does :

- Sequential Memory

→ LSTM's gated cell state lets it learn to store and forget information as needed, preserving long-range dependencies over 50+ laps.

- Variable-Length Races

→ LSTMs naturally handle sequences of varying length without needing to re-architect the model: you just feed in as many time-steps as that race has.

- Data Volume

→ LSTMs with a few layers (e.g. 2×1200) give plenty of capacity without exploding your parameter count or requiring tens of millions of examples.

- Feature Heterogeneity

→ LSTMs accept flat real-valued vectors, so you can concatenate everything without special accommodations.

- Ease of Training & Tuning

→ LSTMs are a mature, well-understood primitive in PyTorch: there's standard support for truncated BPTT, built-in dropout between layers, Xavier init, etc.

Why LSTM Model

Other contenders :

- GRU

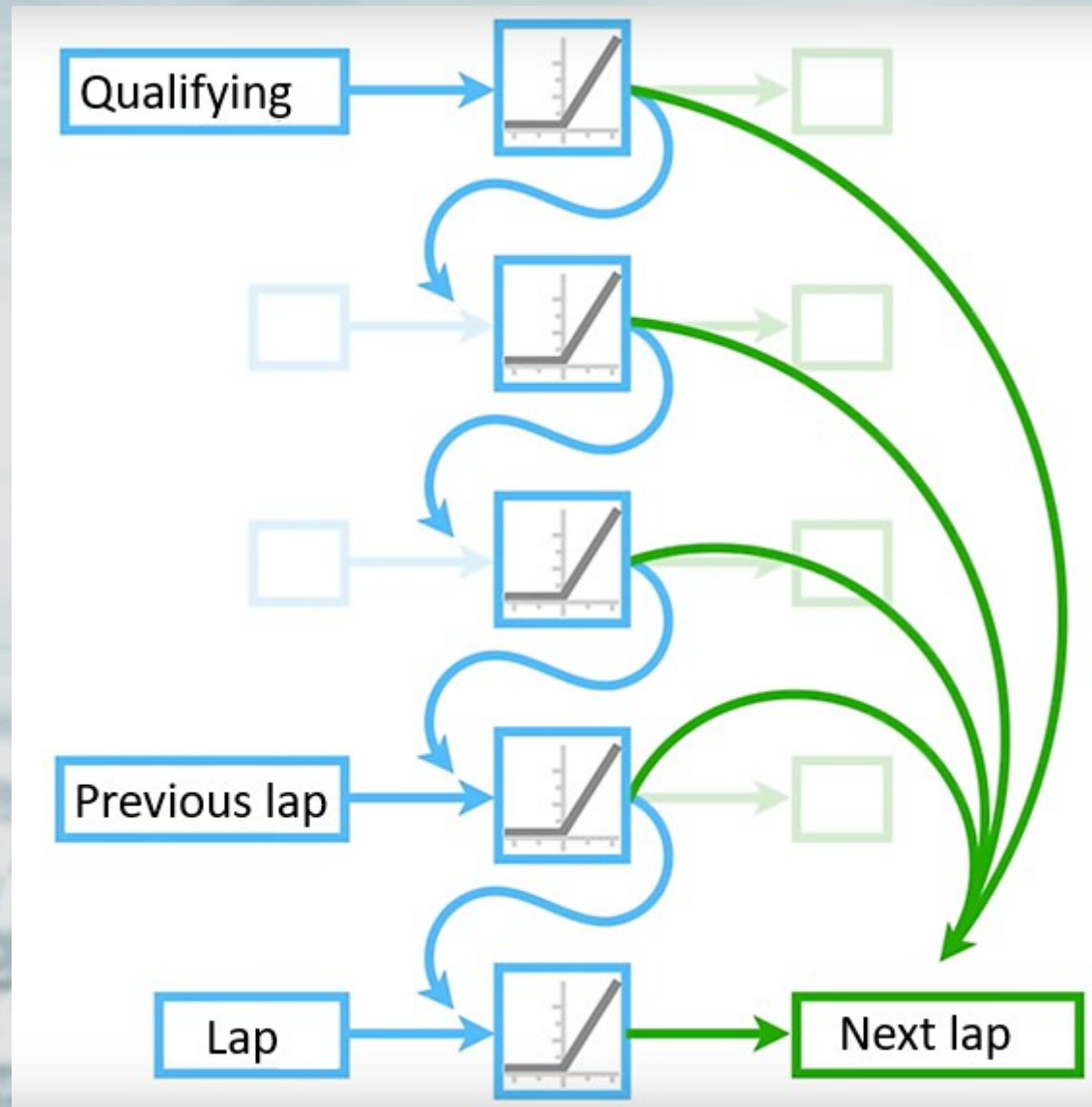
- fewer parameters

- Temporal CNN

- faster parallelizable convolutions

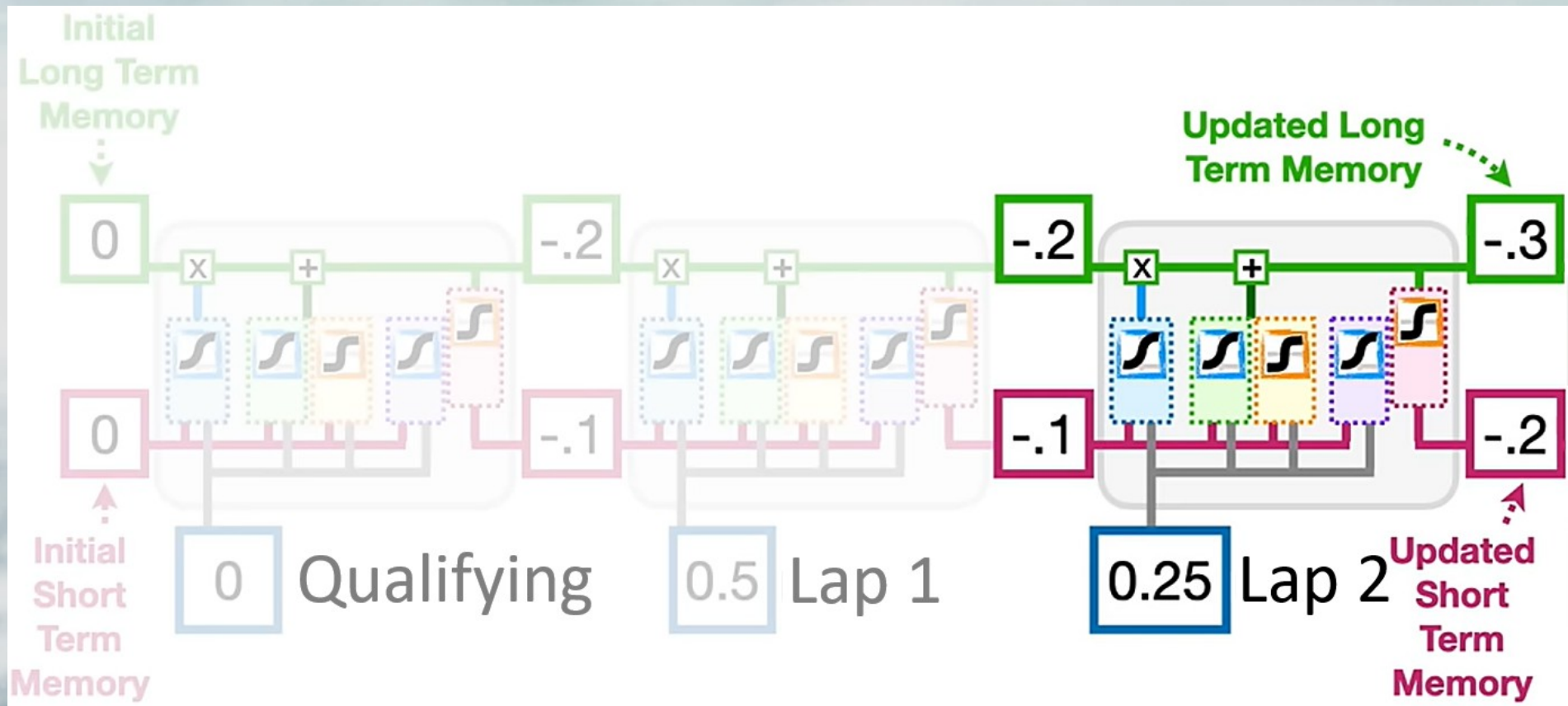
How LSTM works

The principle



How LSTM works

A bit of details



Raw result sample

	code	driver	position	laps till pitting	status	laptime
0	HAM	Lewis Hamilton	1	0.024014	No Problem	101.5
1	VER	Max Verstappen	2	-0.023678	No Problem	101.5
2	BOT	Valtteri Bottas	3	-0.000930	No Problem	100.4
19	LAT	Nicholas Latifi	4	0.002529	No Problem	101.8
3	PER	Sergio Pérez	4	-0.001795	No Problem	101.2
4	RIC	Daniel Ricciardo	5	-0.013361	No Problem	102.3
5	SAI	Carlos Sainz	7	0.003256	No Problem	103.0
9	LEC	Charles Leclerc	8	0.022530	No Problem	102.1
12	RUS	George Russell	9	-0.024174	No Problem	103.0
8	GAS	Pierre Gasly	10	-0.025098	No Problem	103.6
7	NOR	Lando Norris	11	0.021469	No Problem	102.2
6	OCO	Esteban Ocon	12	0.019673	No Problem	101.3
16	GIO	Antonio Giovinazzi	13	0.001355	No Problem	103.6
13	VET	Sebastian Vettel	14	0.022170	No Problem	104.2
14	ALB	Alexander Albon	15	-0.034550	No Problem	103.9
15	GRO	Romain Grosjean	16	0.050112	No Problem	104.3
17	MAG	Kevin Magnussen	17	-0.007714	No Problem	103.9
11	STR	Lance Stroll	17	0.024492	No Problem	100.0
18	RAI	Kimi Räikkönen	19	0.016841	No Problem	105.6
10	KVY	Daniil Kvyat	20	0.022690	No Problem	105.5

User Interface (WIP)

Model Input

Season
2025

Round
1

Total number of laps
60

Randomness factor
0 10 50

Model selection
Optimized for Pit Stop Pre

Predict

Formula One Race Lap-by-Lap Prediction

2025 Round 1
Circuit International de Bahrain

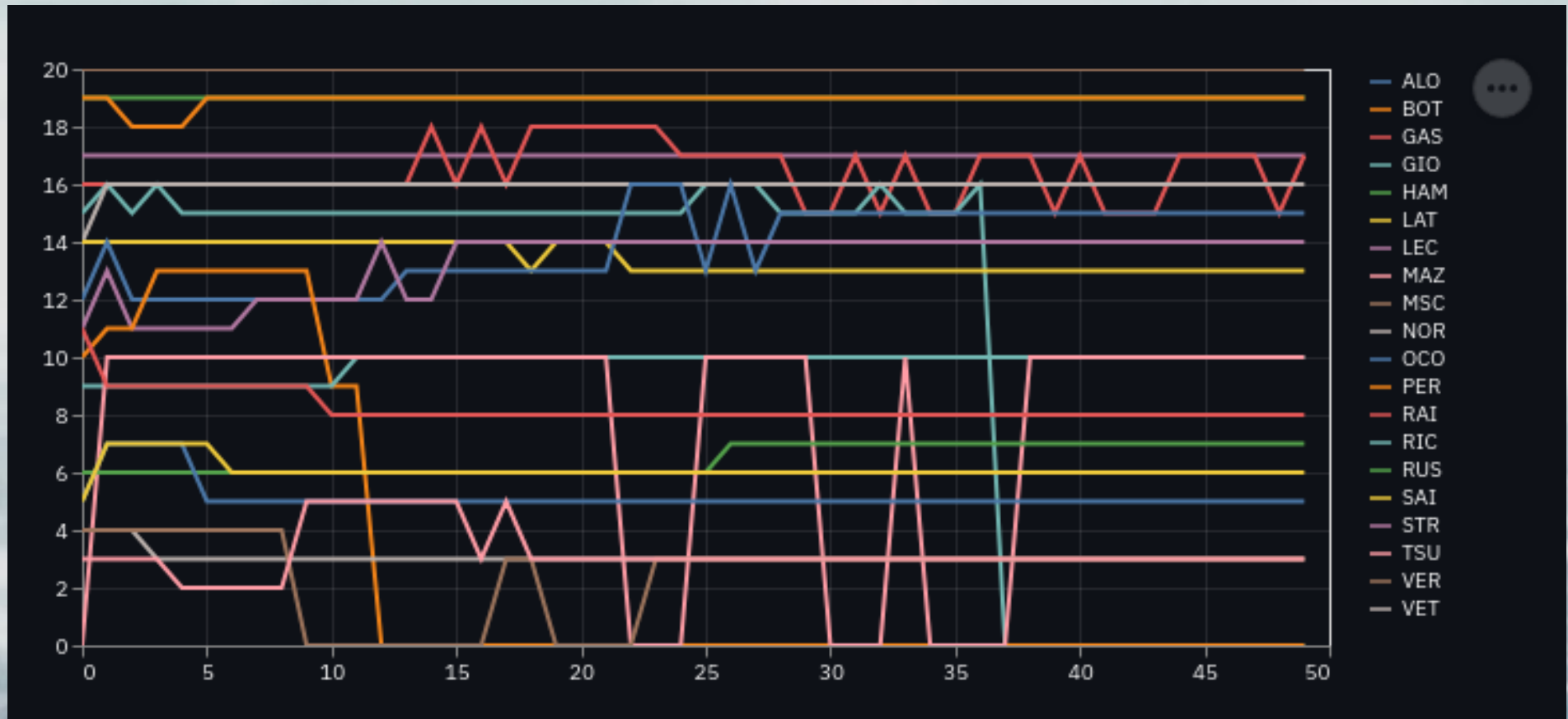
Lap (Slide me to refresh)

1 1 57

Prediction

code	driver	position	laps till pitting	status	laptime
VER	Max Verstappen	1	0	No Problem	1:33.5
LEC	Charles Leclerc	2	0	No Problem	1:39.1
NOR	Lando Norris	3	0	No Problem	1:37.9
OSC	Oscar Piastri	4	5	No Problem	1:37.7
HAM	Lewis Hamilton	5	4	No Problem	1:39.1
RUS	George Russell	6	8	No Problem	1:39.9
SAI	Carlos Sainz	7	10	No Problem	1:33.1
PER	Sergio Perez	8	4	No Problem	1:35.1
ALO	Fernando Alonso	9	1	No Problem	1:30.9
STR	Lance Stroll	10	0	No Problem	1:34.5

Future result layout



Future features

- Integrate weather, tyre compounds, track surface data.
- Model fuel-weight impact when regulations change.
- Explore Transformer-based architectures for improved long-term modeling.